

基于TFJS技术训练的 关键点模型如何在本地部署 ——以“手势分类”为例

钟以维 浙江省三门县教师发展中心

谢作如 浙江省温州科技高级中学

胡家赓 广州中鸣数码科技有限公司

摘要:为解决中小学人工智能通识课中TFJS手势分类模型本地化部署的难题,本研究融合了MediaPipe框架与XEduHub工具链,即利用MediaPipe实现网页和Python环境的关键点检测一致,通过开发标准化XEduHub扩展插件,使初学者仅需四行代码即可将网页训练模型部署至本地。同时,该方案还打通了“网页训练—本地部署”的技术闭环,使无编程基础的学生能完整实践人工智能从数据采集到真实场景应用的全流程,进而提升教学可行性。

关键词: TFJS模型部署;人工智能通识教育;XEduHub;手势分类

中图分类号: G434 **文献标识码:** A **论文编号:** 1674-2117 (2025) 15-0087-03

● 问题的提出: TFJS手势分类模型能否在本地部署

为降低学习难度,多家企业开发了基于网页前端技术的模型训练工具。通过该工具,学生不用编写代码也能亲历数据收集、模型训练和测试的完整环节,因而它在中小学的人工智能通识课中得到了广泛应用。这类工具基本上是采用TensorFlow.js(简称TFJS)实现图像分类、音频分类、关键点分类(手势、姿态)和文本分类等。自浦育团队实现图像分类模型转ONNX后,教师们意识到TFJS模型的训练不仅仅可以用于体验,还可以部署在本地并解决真实问题。因此,

教师们开始关注TFJS类模型训练工具,期待关键点模型也能转换为ONNX,并且支持XEduHub这一工具推理,让中小学的人工智能通识课的内容更加丰富多彩。此外,很多支持Linux的开源硬件已经内置了XEduHub,这样一来,开发一个人工智能作品就会变得简单。然而,关键点分类模型却因自身的技术复杂性,在转换为ONNX格式并实现跨平台部署方面遇到了瓶颈。

● 可行性分析: 基于MediaPipe的关键点识别技术

基于TFJS技术的关键点分类模型训练,一般不是直接采用端到端(End-to-End)方式来实现的,

而是分步实现:先采用关键点检测(Keypoint Detection)获得关键点,再借助全连接神经网络来训练模型。但是,即使将TFJS模型转换成ONNX模型,也不能在本地环境部署使用,因为本地环境也需要使用同样的关键点检测技术获取关键点。想了解这方面的技术细节,请参考文章《用XEduHub推理网页前端训练的声音分类模型》(发表于本刊杂志2025年第9期)。

因此,需要选择一个既支持网页端又支持Python环境运行的关键点检测技术框架,才能做到让TFJS模型可以在本地部署。经调研,MediaPipe凭借其独特的跨

平台特性,成为解决手势分类模型跨平台部署问题的理想选择。它能够同时支持移动端、桌面端、网页(Web)以及边缘设备(如树莓派、行空板等),甚至可部署到嵌入式系统或云端。

1. MediaPipe简介

MediaPipe是由Google Research开发并开源的一款多媒体机器学习模型应用框架,和TFJS一脉相承。在网页端,MediaPipe可以直接借助TFJS实现模型的推理过程。而在Python端,MediaPipe提供了相应的库,接

口简洁易用。经测试,MediaPipe在网页端和Python端的关键点检测结果是一致的。以手势为例,获取的都是21个关键关节坐标。这些关键关节包括指尖、指节、手掌中心等,它们的坐标信息能够准确描述手部的形状和位置。

2. 手势分类模型的训练

在很多人工智能教学平台上,训练一个手势分类模型的过程很简单,类似图像和音频的分类,不需要写任何代码即可训练模型。在训练完成后,平台会提供模型测试功能,直接测试模型的识别效果。

同时,平台直接提供ONNX模型的转换功能,还提供转换后的model.onnx文件及详细的Python推理案例代码。

● 问题的解决:编写一个XEduHub扩展插件

有了ONNX模型和自带的推理文件,还不能直接使用XEduHub进行推理,需要把推理过程封装成repo扩展插件,才能使用XEduHub来进行推理。封装repo插件难度并不高,只要具备一定的Python编程能力,并且拥有模型推理的Python代码,遵循XEduHub的规范,就能自行开发。具体过程一般分为获取Python推理代码、编写封装代码、创建repo仓库、调用repo插件四个步骤。

1. 获取Python推理代码

当在某平台下载ONNX模型时,平台自动执行从TFJS模型到ONNX模型的格式转换,并且提供相应的推理代码。一般来说,推理代码中会提供模型加载并初始化相关的推理引擎的功能,并且完成数据预处理工作。这是因为输入到模型的手势数据需要经过标准化、归一化等处理,以符合模型的训练要求。例如,对于手部关键点坐标数据,需要将其转换为模型能够接受的维度和范围。

2. 编写封装代码

代码封装阶段主要是编写data_process.py文件,该文件将实

```
Python
代码块
#库导入和部分函数略

def inference(image_path): # 推理过程
    # 检查是否需要初始化
    image = cv2.imread(image_path)
    if image is None:
        raise ValueError(f"无法读取图像: {image_path}")

    # 预处理图像
    processed_image = preprocess_image(image, 224, 224)

    # 提取关键点
    keypoints = extract_hand_keypoints(processed_image)
    if keypoints is None:
        return None, {"error": "未检测到手部"}

    # 归一化关键点
    normalized_kps = normalize_keypoints(keypoints)

    # 准备模型输入并推理
    input_data = normalized_kps.reshape(1, -1).astype(np.float32)
    input_name = _session.get_inputs()[0].name
    outputs = _session.run(None, {input_name: input_data})
    predictions = outputs[0][0]

    # 解析预测结果
    class_id = np.argmax(predictions)
    confidence = float(predictions[class_id])
    label = _class_labels[class_id] if class_id < len(_class_labels) else f"未知类别 {class_id}"

    # 输出可视化结果
    result_image = processed_image.copy()
    formatted_result = {
        'class': label,
        'confidence': confidence,
        'class_id': class_id,
        'probabilities': raw_result
    }
    return raw_result, formatted_result
```

图1 核心代码

现模型加载、数据预处理和推理接口等核心功能。推理接口则需要设计得简洁易用,能够接收输入数据并返回推理结果。其核心代码如下页图1所示。

3. 创建repo仓库

将转换后的ONNX模型和编写好的data_process.py等代码文件按照XEduHub的Repo插件规范,上传至魔搭社区(ModelScope),创建专门的repo



图2 repo扩展插件仓库目录结构

```
Python
from XEdu.hub import Workflow as wf
model = wf(repo='cnyawei/Rock-Paper-Scissors')
res, img = model.inference('hand_image.jpg')
print(res)
```

图3 核心代码

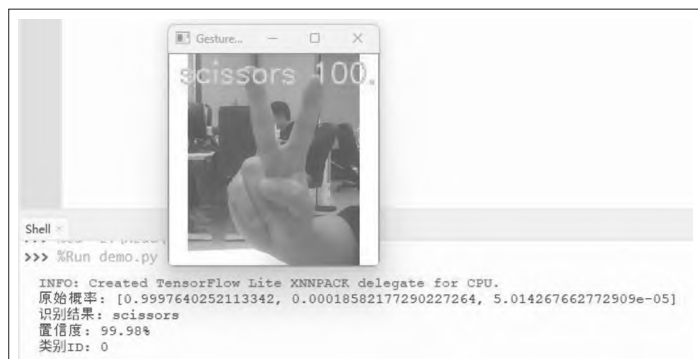


图4 模型推理结果

仓库。在上传过程中,需要填写仓库的相关信息,如仓库名称、描述、标签等,以便其他用户能够方便地查找和使用该插件。同时,要确保仓库中的文件结构清晰,便于XEduHub系统正确识别和加载插件。仓库目录中一般包含如图2所示的文件,其中model.onnx和data_process.py为必需文件。

4. 调用repo插件

在repo扩展插件上传完成后,就可以使用XEduHub工具调用它使用本地算力来进行推理了。最终的推理代码(只有4行)如图3所示。

该代码的关键是wf(repo='cnyawei/Rock-Paper-Scissors'),指定了云端模型的repo仓库名称。在运行代码后,平台会自动下载推理需要的核心文件。model.inference('hand_image.jpg')实现了对"hand_image.jpg"推理。推理结果如图4所示。

● 总结: 人工智能教育需要更多的学习工具

通过将MediaPipe框架与

XEduHub的repo功能相结合,本文实现了网页前端训练的手势分类模型的高效流通和

跨平台部署,为中小学人工智能通识教育提供了从训练到应用的完整链条。XEduHub的repo功能为模型的封装和共享提供了便捷的途径,降低了模型部署的技术门槛;TFJS技术支持在浏览器中直接运行ONNX模型,实现了网页端推理与本地化部署的无缝衔接;Web页面数据采集工具则简化了数据集的构建流程,为模型训练提供了充足的数据支持。

受此启发,笔者针对人工智能通识教育的工具还提出了更多的需求。例如,网页端工具应该提供数据导出功能,导出的数据最好包含原始图片和相应的关键点数据(CSV格式)。又如,既然能实现手势分类,那么姿势分类(基于姿势关键点)、人脸识别(脸部关键点)都能按照同样的方式实现。当然,如果网页端采用的是OnnxRuntime Web技术,那么转换后的模型就能直接用XEduHub内置的代码推理,连MediaPipe环境都不需要安装了。同理,只要有合适的ONNX模型,绝大多数的分类任务都能像图像分类、音频分类、手势分类一样,实现网页前端训练和本地部署的无缝结合。e